

PRASIE LAB1

Introduction to Programming with POSIX Threads

António Barros
June 13, 2026

1 Development Environment Setup

Before you start programming with Pthreads, you must ensure your system has the necessary compiler and library headers installed.

To verify if your system is already configured, open your terminal and check if the compiler is available by running:

- **on Linux:** `gcc --version`
- **on macOS:** `clang --version`

If the command returns version information for the compiler, your environment is likely ready for use.

If the command returns a "command not found" error or prompts you to install a package, you must proceed with the installation steps below. Ensure you select the appropriate instructions for your specific operating system to set up the necessary development toolchain.

1.1 Linux Systems

Most Linux distributions use the GCC (GNU Compiler Collection) suite. You will need to install the essential build packages which include the compiler and standard libraries.

- **Debian / Ubuntu / Mint:**

```
$ sudo apt update  
$ sudo apt install build-essential
```

- **Fedora / RHEL / CentOS:**

```
$ sudo dnf groupinstall "Development Tools"
```

- **Arch Linux:**

```
$ sudo pacman -S base-devel
```

Finally, to verify that your compiler is ready, check the version:

```
$ gcc --version
```

1.2 macOS Systems

macOS uses the Clang compiler, provided by the Xcode Command Line Tools. This is natively integrated with the POSIX threads library. To install or verify the tools, run the following command in your terminal:

```
% xcode-select --install
```

If the tools are already installed, a message will appear stating that the software is already installed. Otherwise, a prompt will appear to download and install them.

Finally, to verify that your compiler is ready, check the version:

```
% clang --version
```

1.3 Compilation

When compiling your code, ensure you link the Pthreads library. Using GCC or Clang, the command syntax is:

- **for gcc:** `gcc -o output_name source_file.c -lpthread`
- **for Clang:** `clang -o output_name source_file.c`

Note

When compiling your Pthreads programs with gcc, always remember to link the POSIX threads library using the `-lpthread` flag. This ensures compatibility and proper library resolution.

Note

If you are using a specific IDE (such as CLion, VS Code, or Eclipse), ensure you add `pthread` to your project's linker settings or `CMakeLists.txt` to avoid *"undefined reference to pthread_create"* errors.

2 Pthreads Quick Reference

This cheat sheet provides a summary of the most essential POSIX threads functions. You will frequently refer to these when developing your multi-threaded applications.

Function	Description
<code>pthread_create()</code>	Creates a new thread and starts its execution.
<code>pthread_exit()</code>	Terminates the calling thread.
<code>pthread_join()</code>	Suspends the calling thread until the target thread terminates.
<code>pthread_self()</code>	Returns the unique identifier of the calling thread.
<code>pthread_equal()</code>	Compares two thread identifiers for equality.

Table 1: Basic Pthreads management functions

3 Programming Exercises

This section contains three practical exercises designed to introduce you to Pthreads. You should implement these in order, ensuring you understand the memory management implications of each step.

3.1 Exercise 1: Thread Identification

Your task is to implement a program that spawns five worker threads. The implementation must follow these requirements:

- **Worker Threads:** Each worker thread should print a “hello” message to the console, including its unique identifier obtained via `pthread_self()`.
- **Main Thread:**
 - For each thread spawned, the main thread must print the identifier of the newly created thread.
 - After spawning all workers, the main thread must wait for them to complete using `pthread_join()` before printing a “goodbye” message and terminating.

Analysis:

1. Compile and run the program. Is the output order deterministic? Run the program repeatedly to observe whether the output follows a consistent pattern.
2. Explain why the `pthread_join()` loop ensures the main thread waits for all worker threads to complete before finishing its execution.
3. Comment out the `pthread_join()` calls and observe the behaviour of the worker threads when the `main()` function returns. Explain the underlying cause of your observations.

3.2 Exercise 2: The Loop Index Pitfall

A common error when passing data to threads is passing the address of a local variable that is modified in a loop, such as an iteration index. Implement a program that creates threads within a loop and passes the current loop index `i` to the worker function.

1. Initially, pass the address of the loop index (`&i`) directly as the thread argument. You will likely observe that the threads fail to print the expected sequence of values taken by `i`. This is a classic concurrency bug caused by the main thread modifying the memory location before the worker threads have a chance to read it.
2. To resolve this, declare an array as a local variable in the `main()` function. Assign the value of `i` to an array cell and pass the address of that unique cell to the corresponding thread. *Critically reflect:* Is this approach safe if the main thread terminates before the worker threads?
3. Modify your code to declare the array as a `static` global variable instead. Verify that your code functions correctly and consider the implications for data isolation.

3.3 Exercise 3: Structure Encapsulation

In more complex scenarios, you often need to pass multiple parameters to a thread. This exercise explores how to achieve this using structures.

Implement a program that uses a shared global array of 3 integers. All elements of this array should be initially set to zero.

You must spawn three threads. Each thread performs the following logic in a loop:

1. Read the current value of a specific input cell.
2. Immediately set that cell to zero.
3. Add a constant value to the read number and store the result in a specified output cell.
4. Wait for a defined number of seconds before the next iteration.

The argument passed to each thread must be a pointer to a struct containing:

- The index of the input cell.
- The index of the output cell.
- The constant value to add.
- The delay (in seconds) between iterations.

After spawning the threads, the main thread should print the values of the shared array every second to observe the updates.

3.4 Exercise 4: Monitoring and Alert Systems

In this exercise, you will implement a multi-threaded system where one thread simulates a sensor (modifying state) and the second acts as a monitor (observing state).

Implement a program featuring a global integer variable, initialised to 20, which represents a system parameter (e.g., temperature). Two threads must be spawned:

- **Sensor Thread:** Enters an infinite loop where it adds a random value (between -3 and 3) to the global variable. After each modification, it sleeps for a random interval between 1 and 5 seconds.
- **Monitor Thread:** Inspects the global variable every second. It must perform the following logic:
 - Print the current value.
 - If the value exceeds 35, print: "WARNING: OVERHEATING DETECTED".
 - If the value drops below 0, print: "ALERT: FREEZING HAZARD".
 - If 3 consecutive alerts occur, print: "CRITICAL SYSTEM FAILURE: CONTINUOUS ALERT STATE". This message should persist until the value returns to the safe range (0 to 35).

After spawning both threads, the main thread should terminate immediately without waiting for the others.

Note

To generate a random integer, use the `rand()` function, which returns a value between 0 and `RAND_MAX`. To generate a random number within a specific interval `[MIN_VALUE, MAX_VALUE]`, you must determine:

- **Number of possible values:** `NVALUES = (MAX_VALUE - MIN_VALUE + 1)`
- **Minimum value:** `MIN_VALUE`

The code to generate a number in this interval is:

```
int random_number = (rand() % NVALUES) + MIN_VALUE;
```

If you have any questions regarding this arithmetic, please consult your instructor.

4 Concurrency Pitfalls and Race Conditions

In this exercise, you will investigate how concurrent access to shared memory leads to non-deterministic behaviour and data corruption when not properly synchronised.

Problem 1: The Counter Race Condition Implement a program with a shared global variable counter, initialised to 0. Spawn 3 threads, where each thread executes a loop that increments the counter 100 000 times.

1. After joining the threads, print the final value of the counter.
2. Does the result match the expected value of 300,000? Run the program several times.
3. Explain why the increment operation (`counter++`) is not atomic and how it causes the race condition.

Problem 2: The Inconsistent State (The Lost Update) Consider a system where you have an account balance represented by a global variable `balance = 1000`.

- Create a *depositor* thread that adds 100 to the balance 500 times.
- Create a *withdrawer* thread that subtracts 100 from the balance 500 times.

- Add a short `usleep()` (e.g., 100 microseconds) between reading the balance and writing the new value to simulate a complex operation.
1. What is the expected final balance? Does your program consistently reach that value?
 2. Explain the consequence of context switching occurring between the *read* and *write* operations.
 3. How does the introduction of `usleep()` impact the probability of encountering an error?